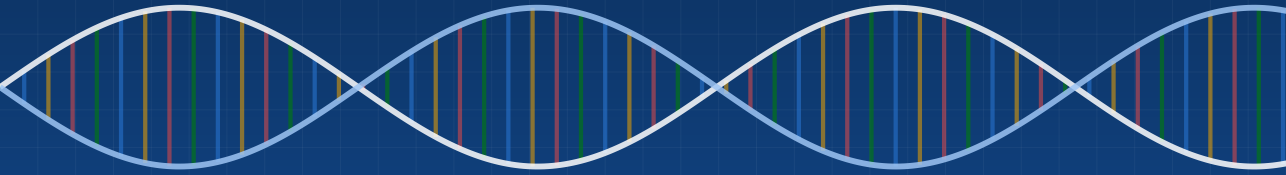


Preparación del laboratorio digital



Un artículo sobre ciencia computacional en una publicación científica no es la obra académica en sí misma; es solo la publicidad de esa obra. La verdadera obra es el entorno completo de desarrollo de software y el conjunto completo de instrucciones que generaron las figuras.

— Buckheit y Donoho (1995), resumiendo una idea de Jon Claerbout

● Al terminar este capítulo usted podrá

- Representar secuencias biológicas como arreglos en memoria, estimar el costo de un cálculo con notación asintótica y explicar por qué la vectorización con numpy es órdenes de magnitud más rápida que un bucle.
- Reconocer los límites de la aritmética de punto flotante y usar el espacio logarítmico para trabajar con probabilidades diminutas.
- Organizar datos en tablas ordenadas (*tidy*) con pandas y construir figuras que respeten los principios de la percepción visual.
- Leer, traducir y verificar un genoma real con Biopython y combinar Python con herramientas de línea de comandos como BLAST+.

- Explicar cómo funcionan un generador de números pseudoaleatorios, una función *hash* y un repositorio de Git, y usar los tres para que un análisis sea reproducible.

Un biólogo molecular no empieza su primer experimento sin saber calibrar una micropipeta, preparar un tampón o anotar un protocolo en su cuaderno. El bioinformático tampoco. Su mesa de trabajo es una computadora; sus reactivos, archivos de secuencias; sus instrumentos, bibliotecas de programación y programas de línea de comandos.

Este capítulo prepara ese laboratorio. No es un tutorial de programación: queremos entender *qué ocurre dentro de la máquina* cuando analizamos datos biológicos, porque de ello dependen la velocidad, la exactitud y la credibilidad de todo lo que haremos en el resto del libro. Avanzaremos en tres pasos. Primero convertiremos el cuaderno de Colab en un instrumento de medida: veremos cómo se representan las secuencias en memoria, cuánto cuesta un cálculo, qué precisión tienen los números de la máquina y cómo se organiza y se muestra un conjunto de datos (sección 0.1). Después incorporaremos las herramientas de la disciplina: Biopython, las bases de datos del NCBI y la línea de comandos, con un genoma real, el de SARS-CoV-2, como material de trabajo (sección 0.2). Por último, estudiaremos la maquinaria de la reproducibilidad: los números aleatorios que no lo son, las huellas digitales de los archivos, el control de versiones y los principios que la comunidad ha acordado para que un análisis pueda verificarse (sección 0.3).

0.1 Colab como laboratorio: Python, numpy, pandas y la visualización científica

 Colab · 0.1 · Colab como laboratorio

 [abrir notebook](#)

Un cuaderno que se ejecuta solo

En el laboratorio húmedo, el cuaderno contiene tres cosas mezcladas: el protocolo (lo que pensábamos hacer), las notas (lo que hicimos de verdad) y los resultados pegados con cinta adhesiva (la foto del gel, la curva de crecimiento). Si seis meses después alguien pregunta cómo se obtuvo una banda, la respuesta está en el cuaderno, junto a la banda. Ahora imagine un cuaderno en el que el

protocolo es el experimento: cada vez que alguien lo lee de principio a fin, el experimento vuelve a ocurrir y la foto del gel se regenera ante sus ojos. Eso es un *notebook* computacional.

0.1.1 La computación como instrumento de laboratorio

Un *notebook* de Jupyter, y su variante alojada en la nube, Google Colab, es un documento formado por *celdas*. Las celdas de texto contienen la narración, con ecuaciones y enlaces; las celdas de código contienen instrucciones que se envían a un proceso independiente, el *núcleo* (*kernel*), que las ejecuta y devuelve el resultado para que se muestre debajo de la celda. Kluyver et al. (2016) describen este diseño como un formato de publicación para flujos de trabajo computacionales reproducibles: el mismo archivo documenta, ejecuta y muestra.

La separación entre documento y núcleo tiene una consecuencia que conviene comprender desde el principio. El núcleo guarda en memoria un *estado*: todas las variables, funciones y módulos que se han definido hasta el momento. Ese estado depende del *orden en que se ejecutaron* las celdas, no del orden en que aparecen en la página. Si ejecuta la celda 5, luego corrige la celda 2 y la vuelve a ejecutar, el núcleo contiene una mezcla que no corresponde a ninguna lectura lineal del documento (figura 0.1). Los números entre corchetes a la izquierda de cada celda, [1], [2], ..., registran precisamente ese orden de ejecución.

Colab añade una segunda capa: el núcleo corre en una máquina virtual prestada que se destruye tras un rato de inactividad, llevándose consigo todo lo no guardado y las bibliotecas instaladas. Esa fragilidad obliga a escribir cuadernos que reconstruyan su entorno desde cero, que es justo lo que exige la reproducibilidad (sección 0.3).

El estado oculto

Antes de compartir un cuaderno o de confiar en sus resultados, use *Entorno de ejecución* → *Reiniciar y ejecutar todo*. Si el cuaderno falla o produce otros números, el resultado anterior dependía de un estado que no está escrito en ninguna parte.

0.1.2 Una secuencia es un arreglo de símbolos

La información genética de una molécula de ADN depende solo del orden de sus bases. Para la computadora, por lo tanto, una secuencia es una cadena finita de símbolos.

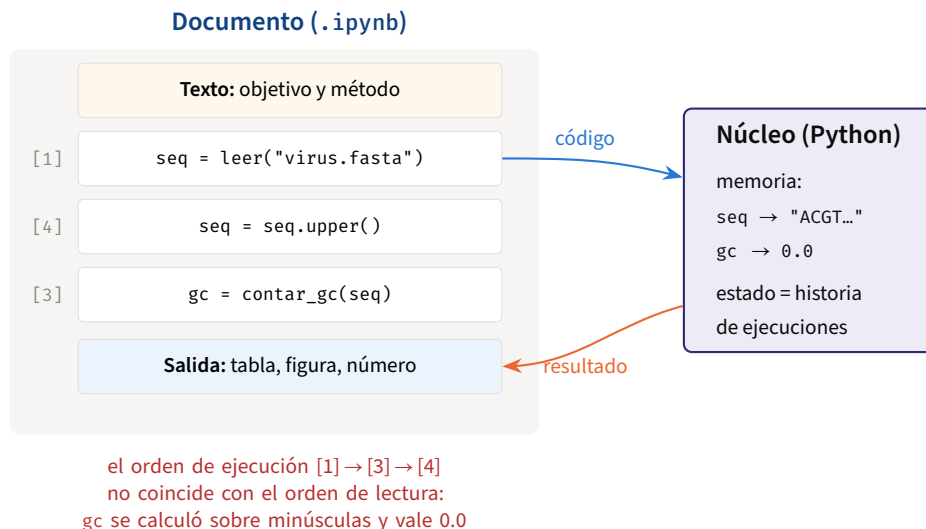


Figura 0.1. Anatomía de un *notebook*. El documento (izquierda) alterna celdas de texto, de código y de salida; el núcleo (derecha) ejecuta el código y conserva en memoria todas las variables. Los contadores [1], [3], [4] revelan un *estado oculto*: la celda que calcula `gc` se ejecutó antes que la que convierte la secuencia a mayúsculas, así que contó cero bases G o C en un archivo escrito en minúsculas. La memoria contiene ya la secuencia corregida, pero `gc` conserva el valor erróneo, distinto del que obtendría un lector que ejecutara el cuaderno de arriba abajo.

Definición 0.1. (Alfabeto, secuencia y codificación)

Un *alfabeto* Σ es un conjunto finito de símbolos; para el ADN, $\Sigma = \{A, C, G, T\}$ y para las proteínas, los 20 aminoácidos. Una *secuencia* de longitud n es una función $x : \{0, 1, \dots, n-1\} \rightarrow \Sigma$, que escribimos $x = x_0x_1 \dots x_{n-1}$. Una *codificación* es una función inyectiva $c : \Sigma \rightarrow \{0, 1\}^b$ que asigna a cada símbolo una palabra de b bits.

Hemos numerado las posiciones desde cero, como Python, y no desde uno, como los biólogos. Esta diferencia es fuente de incontables errores, y conviene fijar la regla de conversión. En Python, el corte $x[i:j]$ contiene las posiciones $i, i+1, \dots, j-1$: es un intervalo *semiabierto* $[i, j)$ cuya longitud es simplemente $j-i$. Un gen anotado en las posiciones biológicas $a..b$ (ambas incluidas, numeradas desde 1) se extrae así:

$$\text{posiciones biológicas } [a, b] \longleftrightarrow x[a-1:b], \quad \text{longitud} = b - a + 1. \quad (0.1)$$

DONDE

a, b	Primera y última posición del segmento en la numeración biológica (base 1, extremos incluidos).
$x[a-1:b]$	Corte de Python equivalente: empieza en el índice $a-1$ y se detiene <i>antes</i> del índice b .

¿Cuánta memoria ocupa una secuencia? Si cada símbolo se codifica con b bits, una secuencia de n símbolos ocupa

$$M(n) = \frac{nb}{8} \text{ bytes,} \quad b \geq \lceil \log_2 |\Sigma| \rceil. \quad (0.2)$$

DONDE

$M(n)$	Memoria necesaria, en bytes (1 byte = 8 bits).
b	Bits por símbolo de la codificación elegida.
$ \Sigma $	Tamaño del alfabeto; $\lceil \log_2 \Sigma \rceil$ es el mínimo número de bits que distingue todos sus símbolos.

Con cuatro bases bastan $\log_2 4 = 2$ bits, pero el texto ASCII usa 8. Para un genoma humano de $3,1 \times 10^9$ bases, la ecuación (0.2) da 775 MB empaquetado a 2 bits (formato `.2bit`), 3,1 GB como texto (`str` o `np.uint8`), 24,8 GB como lista de Python (un puntero de 8 bytes por base) y 99,2 GB como codificación *one-hot* en `float64`, habitual en aprendizaje automático: la misma información, con un factor de 128 de diferencia.

0.1.3 Cuánto cuesta un cálculo: complejidad asintótica

Antes de ejecutar un análisis sobre un genoma completo conviene saber si tardará un segundo o un año. La herramienta para responder sin medir es la notación asintótica, que describe cómo crece el costo de un algoritmo cuando crece el tamaño n de la entrada, sin preocuparse por las constantes que dependen de la máquina.

Definición 0.2. (Notación O grande)

Sean f y g funciones positivas de n . Decimos que $f(n) = O(g(n))$ si existen constantes $c > 0$ y n_0 tales que

$$f(n) \leq c g(n) \quad \text{para todo } n \geq n_0. \quad (0.3)$$

Si además $f(n) \geq c' g(n)$ para alguna $c' > 0$, escribimos $f(n) = \Theta(g(n))$: f y g crecen al mismo ritmo.

DONDE

$f(n)$	Costo real del algoritmo (operaciones o segundos) para una entrada de tamaño n .
$g(n)$	Función de referencia sencilla: n , $n \log n$, n^2 , $2^n \dots$
c, c', n_0	Constantes que no dependen de n ; absorben la velocidad de la máquina y los detalles de implementación.

La intuición es sencilla: $O(g)$ ignora las constantes y se queda con la *forma* del crecimiento. Contar el contenido GC recorre la secuencia una vez: es $\Theta(n)$. Comparar todas las posiciones de una secuencia con todas las de otra, como en el alineamiento del ??, es $\Theta(n^2)$. Ordenar n lecturas es $\Theta(n \log n)$. A 10^9 operaciones por segundo, un algoritmo $\Theta(n^2)$ tarda menos de un segundo sobre el genoma de SARS-CoV-2 (3×10^4 bases), unas siete horas sobre el de *E. coli* (5×10^6) y casi tres siglos sobre el humano (3×10^9), mientras que uno lineal tarda, respectivamente, microsegundos, milisegundos y segundos.

0.1.4 Vectorización: la misma operación sobre millones de datos

Python es un lenguaje *interpretado*: en un bucle `for`, el intérprete vuelve a averiguar en cada vuelta qué tipo de objeto tiene delante, dónde está su método de comparación y cómo sumar el resultado. Cada iteración cuesta decenas de nanosegundos, aunque la operación útil (comparar un byte) cueste menos de uno. Llenar una placa de 96 pozos con una micropipeta monocal, pozo por pozo, se parece mucho a eso.

La biblioteca NumPy resuelve el problema con una estructura de datos, el arreglo `ndarray`, y un estilo de programación, la *vectorización* (Harris et al., 2020). Un arreglo es un bloque contiguo de memoria descrito por cuatro atributos: un puntero al primer byte, un tipo de dato (`dtype`) común a todos los elementos, una forma

(shape) y unos pasos (strides) que indican cuántos bytes hay que avanzar para moverse una posición en cada eje. La dirección del elemento (i, j) de una matriz es

$$\text{dir}(i, j) = \text{dir}_0 + i s_0 + j s_1. \quad (0.4)$$

DONDE

dir_0	Dirección de memoria del primer elemento.
s_0, s_1	Pasos (strides) en bytes: cuánto avanzar para cambiar de fila y de columna.
i, j	Índices de fila y columna (desde 0).

Como todos los elementos tienen el mismo tipo y están uno tras otro, una operación como `a == 71` (¿es esta base una G, cuyo código ASCII es 71?) puede ejecutarse en un bucle escrito en C, compilado, que el procesador además puede paralelizar con instrucciones vectoriales (SIMD). La ecuación (0.4) explica también un truco muy útil: transponer una matriz o tomar una de cada tres bases (`a[:,3]`, las primeras posiciones de los codones) no copia datos, solo cambia los pasos. Esas *vistas* son gratuitas.

```
gc_vectorizado.py

import numpy as np

def gc_loop(seq: str) -> float:
    n = 0
    for b in seq:
        if b == "G" or b == "C":
            n += 1
    return n / len(seq)

def gc_numpy(seq: str) -> float:
    a = np.frombuffer(seq.encode("ascii"), dtype=np.uint8)
    is_gc = (a == ord("G")) | (a == ord("C")) # máscara booleana
    return np.count_nonzero(is_gc) / a.size
```

La figura 0.2 compara ambas versiones. Extrapolada al genoma humano, la diferencia es de unos 160 segundos frente a menos de uno. Observe que ambas versiones siguen siendo $\Theta(n)$: la vectorización cambia la constante, no la complejidad.

La vectorización se completa con la *difusión* (*broadcasting*): cuando se combinan arreglos de formas distintas, NumPy repite virtualmente las dimensiones de tamaño 1 sin copiar datos. La expresión `x[:, None] == y[None, :]`, que compara cada base

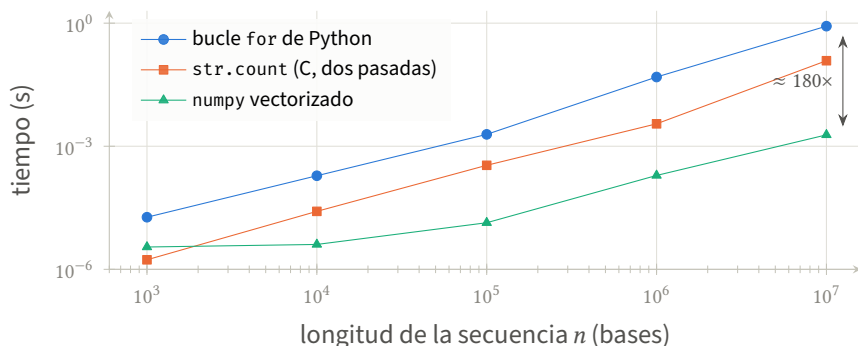


Figura 0.2. Tiempo para calcular el contenido GC de secuencias aleatorias de 10^3 a 10^7 bases (mejor de tres repeticiones, medido con `figuras/cap00/generar.py`). En escala doble logarítmica, un algoritmo $\Theta(n)$ es una recta de pendiente 1: los tres métodos son lineales, como predice el análisis asintótico. Lo que cambia es la constante c de la ecuación (0.3): el bucle interpretado gasta unos 50 ns por base y la versión vectorizada unos 0,3 ns.

de una secuencia con cada base de otra y construye la matriz de un *dot plot* en una línea, es el ejemplo que usaremos en el ??.

0.1.5 Números en la máquina: enteros y punto flotante

Los enteros de la máquina tienen un tamaño fijo: un `np.uint8` solo guarda valores de 0 a 255, y 255 + 1 da 0 sin aviso, como un cuentakilómetros que da la vuelta. Los `int` de Python crecen según haga falta, pero los arreglos de NumPy no.

Los números reales plantean un problema más sutil. Una computadora no puede almacenar π , ni siquiera 0,1, con infinitos decimales. El estándar IEEE 754, que siguen prácticamente todos los procesadores, representa un número de doble precisión (`float64`) con 64 bits: uno de signo, once de exponente y 52 de fracción (Goldberg, 1991):

$$x = (-1)^s \times \left(1 + \sum_{k=1}^{52} f_k 2^{-k} \right) \times 2^{e-1023}. \quad (0.5)$$

DONDE

s	Bit de signo (0 positivo, 1 negativo).
f_k	Bits de la fracción o mantisa; el 1 inicial está implícito (“bit escondido”).
e	Exponente almacenado, un entero de 11 bits; se le resta el sesgo 1023.

La consecuencia fundamental de la ecuación (0.5) es que los números representables no están distribuidos uniformemente: entre dos potencias de 2 consecutivas hay siempre el mismo número de ellos (2^{52}), así que su separación *absoluta* crece con la magnitud mientras su separación *relativa* se mantiene casi constante: entre 1 y 2 los números vecinos distan 2^{-52} , entre 2^{52} y 2^{53} distan exactamente 1, y $2^{53} + 1$ ya no es representable. El número que mide esa separación relativa es el *épsilon de la máquina*, $\varepsilon = 2^{-52} \approx 2,22 \times 10^{-16}$, y de él se deriva el modelo estándar del error de redondeo:

$$\text{fl}(x \circ y) = (x \circ y)(1 + \delta), \quad |\delta| \leq u = \frac{\varepsilon}{2}. \quad (0.6)$$

DONDE

$\text{fl}(\cdot)$	Resultado que devuelve la máquina: el número representable más cercano al valor exacto.
$\circ$	Cualquiera de las operaciones $+$, $-$, $\times$, $\div$.
δ, u	Error relativo cometido y su cota, la unidad de redondeo ($\approx 1,1 \times 10^{-16}$).

La ecuación (0.6) parece inocente, porque un error de 10^{-16} rara vez importa. Hay, sin embargo, tres situaciones en las que muerde. La primera es la comparación exacta: `0.1 + 0.2 == 0.3` es `False`, porque ni 0,1 ni 0,2 tienen representación binaria finita y sus errores de redondeo no se cancelan; por eso se compara con una tolerancia (`np.isclose`). La segunda es la *cancelación catastrófica*: al restar dos números casi iguales, los dígitos correctos se anulan y queda el error, de modo que `1e16 + 1 - 1e16` devuelve 0. La tercera, la más importante en bioinformática, es el *subdesbordamiento* (*underflow*) al multiplicar muchas probabilidades.

Probabilidades diminutas y el espacio logarítmico

La probabilidad de observar una secuencia concreta de 1000 bases bajo un modelo uniforme es $0,25^{1000} \approx 10^{-602}$. El número positivo más pequeño que admite un `float64` es del orden de 5×10^{-324} , así que el producto se redondea a cero exacto,

y con él cualquier razón de verosimilitudes que dependa de él. Los modelos de Márkov, los HMM y la inferencia filogenética del resto del libro multiplican miles de probabilidades; sin un remedio, serían inutilizables.

El remedio es trabajar con logaritmos: $\log \prod_i p_i = \sum_i \log p_i$, y la suma de 1000 términos de $\ln 0,25$ es simplemente $-1386,29$, un número perfectamente ordinario. La dificultad aparece cuando hay que *sumar* probabilidades (por ejemplo, para marginalizar sobre estados ocultos) que solo conocemos por sus logaritmos. La solución es un pequeño teorema que se usa en todas partes.

Teorema 0.1. (Log-suma-exp)

Sean $a_1, \dots, a_k$ números reales (logaritmos de probabilidades) y $m = \max_i a_i$. Entonces

$$\log \sum_{i=1}^k e^{a_i} = m + \log \sum_{i=1}^k e^{a_i - m}, \quad (0.7)$$

y el lado derecho puede evaluarse sin desbordamiento ni subdesbordamiento, porque todos los exponentes son ≤ 0 y al menos uno vale exactamente 0.

DONDE

a_i	Logaritmo de la i -ésima probabilidad, $a_i = \log p_i$.
m	El mayor de los a_i ; se “factoriza” fuera de la suma.

La demostración es una línea: $\sum_i e^{a_i} = e^m \sum_i e^{a_i - m}$, y se toma el logaritmo. El truco está en que la suma de la derecha vale entre 1 y k , un número que la máquina maneja sin problemas.

Por ejemplo, con $\log p_1 = -1000$ y $\log p_2 = -1001$, la evaluación directa (`np.log` de la suma de `np.exp(-1000)` y `np.exp(-1001)`) devuelve $-\infty$, mientras que la ecuación (0.7) da $-1000 + \log(1 + e^{-1}) = -999,68674$; NumPy y SciPy la ofrecen como `np.logaddexp` y `scipy.special.logsumexp`.

0.1.6 Un primer modelo: el contenido GC y la distribución binomial

Con las herramientas anteriores podemos formular nuestra primera medida bioinformática. Los pares G–C se unen con tres puentes de hidrógeno y los pares A–T con dos, de modo que la proporción de G y C influye en la estabilidad térmica del ADN, en el diseño de cebadores y en la eficiencia de la secuenciación. Además,

varía mucho entre especies y dentro de un mismo genoma. El *contenido GC* de una secuencia x es

$$\text{GC}(x) = \frac{n_G + n_C}{n_A + n_C + n_G + n_T}. \quad (0.8)$$

DONDE

$n_A, \dots, n_T$ Número de apariciones de cada base en x ; las ambigüedades como N no cuentan en el denominador.

Al calcular el GC en ventanas deslizantes a lo largo de un genoma, los valores fluctúan. ¿Cuánto de esa fluctuación es biología y cuánto es puro azar de muestreo? Para responder necesitamos un *modelo nulo*. Supongamos que cada base es G o C con probabilidad p , independientemente de las demás. El número de bases GC en una ventana de W posiciones sigue entonces una distribución binomial, y la proporción observada tiene media y varianza

$$K \sim \text{Bin}(W, p), \quad \mathbb{E}\left[\frac{K}{W}\right] = p, \quad \text{Var}\left[\frac{K}{W}\right] = \frac{p(1-p)}{W}. \quad (0.9)$$

DONDE

K Número de bases G o C en la ventana (variable aleatoria).
 W Tamaño de la ventana, en bases.
 p Probabilidad de que una base sea G o C; se estima con el GC del genoma completo.

La desviación estándar $\sqrt{p(1-p)/W}$ decrece como $1/\sqrt{W}$: ventanas cuatro veces más grandes reducen el ruido a la mitad, a costa de perder resolución espacial. Por el teorema central del límite, alrededor del 95 % de las ventanas deberían caer dentro de $p \pm 2\sqrt{p(1-p)/W}$ si el modelo nulo fuera cierto.

Ejemplo 0.1. (El GC de SARS-CoV-2 frente a su modelo nulo)

El genoma de referencia de SARS-CoV-2 (NC_045512.2) tiene 29 903 bases, de las cuales 11 355 son G o C: $p = 0,3797$. Con ventanas de $W = 500$ bases,

$$\sigma = \sqrt{\frac{0,3797 \times 0,6203}{500}} = 0,0217,$$

de modo que el modelo binomial predice que casi todas las ventanas tendrán un GC entre 0,336 y 0,423. En el genoma real (figura 0.3), el GC por ventana

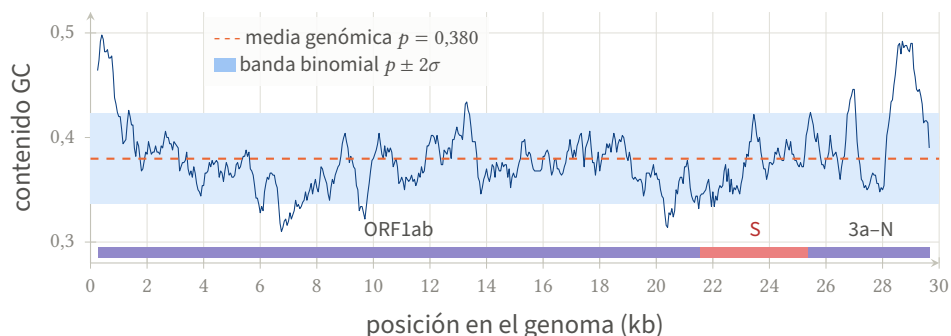


Figura 0.3. Contenido GC en ventanas de 500 bases (paso de 50) a lo largo del genoma de referencia de SARS-CoV-2. La banda azul es la región $p \pm 2\sigma$ que predice el modelo binomial (ecuación (0.9)); si el genoma fuera homogéneo, solo un 5 % de las ventanas escaparía de ella. Escapa casi el triple. Los extremos, sobre todo hacia el final del genoma (genes estructurales y regiones no traducidas), revelan heterogeneidad biológica real.

oscila entre 0,310 y 0,498, y el 14,4 % de las ventanas cae fuera de la banda de $\pm 2\sigma$, casi el triple de lo esperado. El exceso de variación (*sobredispersión*) nos dice que el genoma no es una sucesión de monedas idénticas: su composición cambia de una región a otra.

Calcular el GC de cada ventana por separado costaría $\Theta(nW)$. Un truco clásico lo reduce a $\Theta(n)$: se calcula una vez la suma acumulada $S_i = \sum_{k < i} g_k$ del indicador g_k (1 si la base k es G o C) y el GC de la ventana $[i, i + W)$ es simplemente $(S_{i+W} - S_i) / W$.

gc_ventanas.py

```
import numpy as np
from Bio import SeqIO

rec = SeqIO.read("NC_045512.2.gb", "genbank")
a = np.frombuffer(bytes(rec.seq), dtype=np.uint8)
g = ((a == ord("G")) | (a == ord("C"))).astype(float)
S = np.concatenate([[0.0], np.cumsum(g)]) # sumas acumuladas
W, step = 500, 50
starts = np.arange(0, a.size - W + 1, step)
gc = (S[starts + W] - S[starts]) / W # Theta(n), no Theta(nW)
p = g.mean(); sd = np.sqrt(p * (1 - p) / W)
print(f"p={p:.4f} fuera de 2 sd: {np.mean(abs(gc - p) > 2*sd):.1%}")
```

0.1.7 Datos tabulares: pandas y el formato ordenado

Casi todo resultado bioinformático termina en una tabla: genes con sus coordenadas, muestras con sus metadatos, resultados de BLAST con sus puntuaciones. La biblioteca pandas introdujo en Python el `DataFrame`, una tabla bidimensional cuyas columnas pueden tener tipos distintos y cuyas filas y columnas llevan etiquetas, con operaciones de agrupación, unión y remodelado inspiradas en las bases de datos relacionales y en R (McKinney, 2010).

Tener la herramienta no basta; hay que decidir la *forma* de la tabla. Una misma información puede escribirse de muchas maneras, y algunas facilitan el análisis mientras que otras lo entorpecen. Wickham (2014) formalizó la forma más conveniente con el nombre de datos *ordenados* (*tidy data*).

Definición 0.3. (Datos ordenados)

Un conjunto de datos está ordenado si (1) cada variable forma una columna, (2) cada observación forma una fila y (3) cada tipo de unidad observacional forma una tabla.

Las hojas de cálculo del laboratorio casi nunca cumplen la definición: suelen usar la forma “ancha” de la figura 0.4, con la variable *muestra* escondida en los encabezados. Para agrupar, filtrar o graficar conviene la forma “larga”, una medición por fila; en pandas se pasa de una a otra con `melt` y `pivot`.

Una advertencia: al abrir una lista de genes, Excel convierte por defecto símbolos como *SEPT2* o *MARCH1* en fechas, y Ziemann et al. (2016) hallaron que alrededor de una quinta parte de los artículos de genómica con listas de genes en Excel contenían estas conversiones erróneas. Lea y escriba las tablas con código y trate los archivos originales como de solo lectura.

0.1.8 Principios de visualización científica

Una figura es un instrumento de medida para el ojo. Como cualquier instrumento, tiene una resolución, y esa resolución no es la misma para todas las magnitudes visuales. Cleveland y McGill (1984) lo midieron experimentalmente: pidieron a observadores que estimaran proporciones codificadas de distintas maneras y ordenaron las codificaciones según la exactitud de los juicios. De más a menos precisas: posición sobre una escala común; posición en escalas no alineadas; longitud, dirección y ángulo; área; volumen y curvatura; y, por último, sombreado y saturación del color.

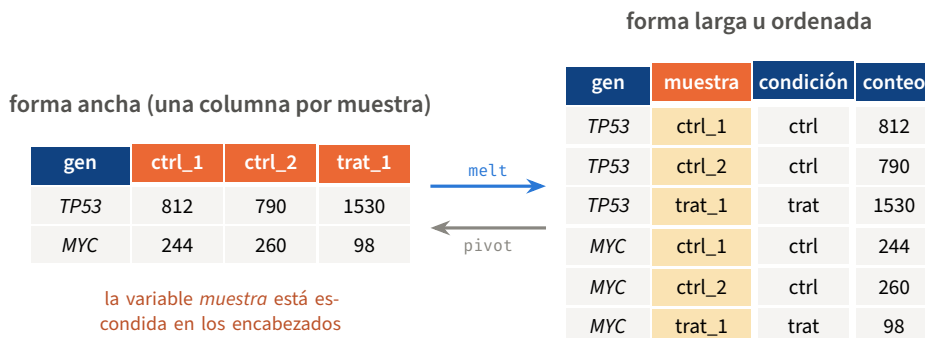


Figura 0.4. La misma tabla de conteos de expresión en dos formas. En la forma ancha (izquierda) los nombres de las muestras son encabezados; en la forma ordenada (derecha) son valores de una columna, cada fila es una observación y aparece una variable nueva, la condición, derivada de la muestra. Todas las operaciones de agrupación y de gráficas de pandas, seaborn o plotly esperan la forma ordenada.

La jerarquía justifica reglas prácticas que usaremos en todo el libro: preferir barras o puntos a gráficas de pastel (que codifican con ángulos y áreas), reservar el color para distinguir categorías o para magnitudes secundarias, y comparar lo que importa comparar sobre un eje común. Tufte (2001) añadió un principio de economía: la *razón datos-tinta*,

$$R = \frac{\text{tinta dedicada a los datos}}{\text{tinta total de la figura}}, \quad (0.10)$$

debe ser tan alta como sea razonable. Rejillas gruesas, fondos sombreados, efectos tridimensionales y leyendas redundantes consumen tinta sin aportar información; Tufte los llamó *chartjunk*. En la práctica, esto se traduce en rotular directamente las series en lugar de usar leyendas, en rejillas tenues y en títulos que enuncian la conclusión.

El color como dato

Cuando el color codifica una magnitud continua (una matriz de expresión, un mapa de contactos), el mapa de color debe ser *perceptualmente uniforme*: pasos iguales en el dato deben producir pasos iguales en la sensación visual. La luminosidad L^* del espacio CIELAB es una buena medida de esa sensación. El mapa arcoíris *jet*, durante años el predeterminado de muchos programas, viola esta condición: su luminosidad sube y baja de forma irregular, de modo que crea bordes donde el dato

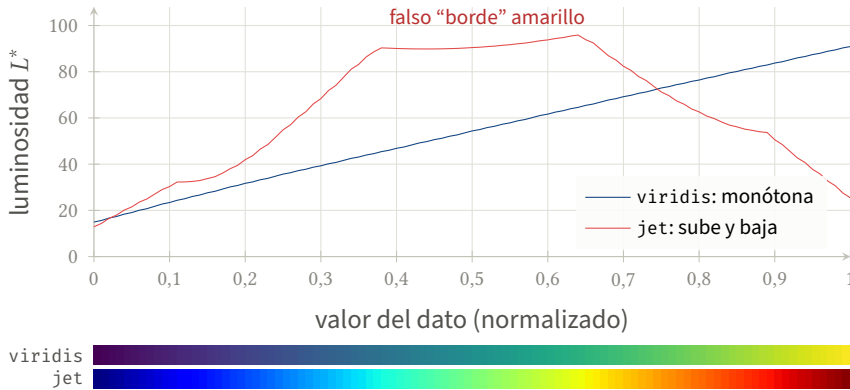


Figura 0.5. Luminosidad perceptual L^* (CIELAB) a lo largo de dos mapas de color, calculada a partir de sus valores RGB. En viridis (azul) crece de forma monótona y casi lineal: el ojo lee un orden correcto incluso en escala de grises. En jet (rojo) sube hasta el cian y el amarillo y vuelve a caer: dos datos distintos pueden verse igual de claros, y la franja amarilla central aparece como un borde que no existe en los datos.

es suave y oculta cambios donde el dato varía (figura 0.5). Además, es ilegible para las personas con deficiencias en la visión del color, que confunden sus rojos y verdes. Cramer et al. (2020) documentaron lo extendido de este mal uso y recomiendan mapas derivados científicamente, como viridis o cividis.

Todas las figuras de este libro, y las del curso en Colab, se construyen con estos criterios. En Python la herramienta básica es matplotlib (Hunter, 2007), sobre la que se apoyan bibliotecas de más alto nivel; para figuras interactivas usaremos plotly en los cuadernos.

0.2 Biopython y herramientas de línea de comandos

 Colab · 0.2 · Biopython y herramientas

 abrir notebook

Instrumentos calibrados por otros

Nadie fabrica su propia micropipeta antes de cada experimento. Se compra un instrumento calibrado, probado por miles de usuarios, y se dedica el esfuerzo a la pregunta biológica. En la sección anterior escribimos a mano

funciones para contar bases; fue útil para entender qué ocurre por dentro. A partir de aquí usaremos dos tipos de instrumentos ya calibrados: las bibliotecas de Python, que son como la gaveta de pipetas, puntas y gradillas de la mesa de trabajo, y los programas de línea de comandos, que son los equipos grandes del laboratorio (el termociclador, el secuenciador), especializados, muy rápidos y controlados mediante instrucciones escritas.

Biopython es la biblioteca de referencia para biología molecular en Python: un conjunto de herramientas libres, desarrollado por una comunidad internacional, que va de la manipulación de secuencias y la lectura de decenas de formatos al acceso programático a bases de datos (Cock et al., 2009). Aquí la usaremos para ir del ADN a la proteína, descargar y verificar un genoma real y conectar Python con programas externos.

0.2.1 El objeto Seq: complemento y complemento reverso

Un objeto Seq se comporta como una cadena de texto, pero “conoce” la biología: sabe complementar, transcribir y traducir. La operación más frecuente, y la que más errores provoca, es el complemento reverso. Las dos hebras de la doble hélice son *antiparalelas*: una corre en sentido $5' \rightarrow 3'$ y la otra en sentido $3' \rightarrow 5'$. Como por convención toda secuencia se escribe de $5'$ a $3'$, leer la hebra opuesta exige complementar cada base e invertir el orden.

Definición 0.4. (Complemento reverso)

Sea $\kappa : \Sigma \rightarrow \Sigma$ el complemento de Watson y Crick, $\kappa(A) = T$, $\kappa(T) = A$, $\kappa(C) = G$, $\kappa(G) = C$. El complemento reverso de $x = x_0 \cdots x_{n-1}$ es la secuencia $\bar{x}$ de la misma longitud con

$$\bar{x}_i = \kappa(x_{n-1-i}), \quad i = 0, \dots, n-1. \quad (0.11)$$

DONDE

κ	Función de complemento base a base.
$\bar{x}_i$	Base en la posición i del complemento reverso; se toma de la posición simétrica $n-1-i$ de x .

De la definición se sigue que el complemento reverso es una *involución*: aplicado dos veces devuelve la secuencia original, $\bar{\bar{x}} = x$, porque tanto κ como la inversión del orden lo son. Una secuencia que coincide con su complemento reverso, $\bar{x} = x$, se llama *palindrómica*; muchos sitios de restricción lo son, como GAATTC (EcoRI), porque las enzimas que los cortan actúan como dímeros simétricos.

0.2.2 El código genético como función

La traducción es una función que lee la secuencia de tres en tres. ¿Por qué tres? Porque con palabras de k letras sobre un alfabeto de cuatro se pueden escribir 4^k palabras distintas, y hacen falta al menos 21 (veinte aminoácidos y una señal de parada): $4^1 = 4$ y $4^2 = 16$ no alcanzan, mientras que $4^3 = 64$ sobran. El código genético estándar es, por lo tanto, una función sobreyectiva

$$\tau : \{A, C, G, U\}^3 \longrightarrow \{20 \text{ aminoácidos}\} \cup \{*\}, \quad |\text{dominio}| = 64, \quad |\text{imagen}| = 21. \quad (0.12)$$

DONDE

τ	Tabla de traducción; Biopython la guarda en <code>Bio.Data.CodonTable</code> (la estándar es la tabla 1 del NCBI).
$\{\dots\}^3$	Conjunto de todos los codones, palabras de tres nucleótidos.
$*$	Señal de parada (UAA, UAG, UGA).

Como hay más codones que significados, el código es *degenerado*: varios codones sinónimos codifican el mismo aminoácido. La figura 0.6 muestra que la degeneración no es aleatoria. Los sinónimos suelen diferir solo en la tercera base, de modo que muchas mutaciones en esa posición son silenciosas, y los aminoácidos químicamente parecidos ocupan casillas vecinas, lo que amortigua el efecto de los errores de lectura. Leucina, serina y arginina tienen seis codones cada una; metionina y triptófano, uno solo. AUG, el codón de la metionina, es además la señal de inicio más común.

0.2.3 Seis marcos de lectura y la estadística de los ORF

El ADN no tiene espacios entre codones. Una secuencia puede leerse empezando en la primera, la segunda o la tercera base, y lo mismo en su complemento reverso: hay seis *marcos de lectura*. En general solo uno de ellos, en cada región, corresponde a la proteína real. Un *marco abierto de lectura* (ORF, *open reading frame*) es un tramo de codones sin paradas; los genes que codifican proteínas son ORF largos, y buscar ORF largos fue durante décadas la primera estrategia de anotación de genomas.

		2.ª base					
		U	C	A	G		
1.ª base	U	UUU PheF	UCU SerS	UAU TyrY	UGU CysC	U	3.ª base
		UUC PheF	UCC SerS	UAC TyrY	UGC CysC	C	
		UUA LeuL	UCA SerS	UAA Stop*	UGA Stop*	A	
		UUG LeuL	UCG SerS	UAG Stop*	UGG TrpW	G	
	C	CUU LeuL	CCU ProP	CAU HisH	CGU ArgR	U	
		CUC LeuL	CCC ProP	CAC HisH	CGC ArgR	C	
		CUA LeuL	CCA ProP	CAA GlnQ	CGA ArgR	A	
		CUG LeuL	CCG ProP	CAG GlnQ	CGG ArgR	G	
	A	AUU IleI	ACU ThrT	AAU AsnN	AGU SerS	U	
		AUC IleI	ACC ThrT	AAC AsnN	AGC SerS	C	
		AUA IleI	ACA ThrT	AAA LysK	AGA ArgR	A	
		AUG MetM	ACG ThrT	AAG LysK	AGG ArgR	G	
	G	GUU ValV	GCU AlaA	GAU AspD	GGU GlyG	U	
		GUC ValV	GCC AlaA	GAC AspD	GGC GlyG	C	
		GUA ValV	GCA AlaA	GAA GluE	GGA GlyG	A	
		GUG ValV	GCG AlaA	GAG GluE	GGG GlyG	G	

Figura 0.6. El código genético estándar (tabla 1 del NCBI), generado a partir de Bio.Data.CodonTable. Cada bloque corresponde a una primera base (filas) y una segunda base (columnas); dentro del bloque, la tercera base recorre U, C, A, G. Colores: hidrofóbicos (amarillo), polares sin carga (verde azulado), con carga positiva (azul), con carga negativa (rojo), especiales G, P y C (violeta) y paradas (gris). Los bloques casi monocromáticos de la columna U (hidrofóbicos) y la vecindad de D y E muestran que el código agrupa los aminoácidos por química.

¿Qué significa “largo”? Una vez más, necesitamos un modelo nulo. Si las bases fueran independientes con frecuencias p_A, p_C, p_G, p_T , la probabilidad de que un codón cualquiera sea de parada sería

$$\pi = p_T p_A p_A + p_T p_A p_G + p_T p_G p_A = p_T p_A (p_A + 2p_G), \tag{0.13}$$

sumando las probabilidades de TAA, TAG y TGA. El número L de codones consecutivos sin parada sigue entonces una distribución geométrica: cada codón es un “intento” que termina el ORF con probabilidad π . Así,

$$\mathbb{P}(L \geq k) = (1 - \pi)^k, \quad \mathbb{E}[L] = \frac{1 - \pi}{\pi}. \tag{0.14}$$

DONDE

$p_A, \dots, p_T$	Frecuencias de las bases en el genoma (modelo de bases independientes).
π	Probabilidad de que un codón al azar sea de parada.
L	Número de codones de un tramo abierto antes de encontrar una parada.
k	Longitud umbral, en codones.

Ejemplo 0.2. (¿Cuántos ORF largos produce el azar?)

Con bases equiprobables, $\pi = 3/64$ y un ORF aleatorio tiene en promedio $E[L] = 61/3 \approx 20$ codones; la probabilidad de superar 100 codones es $(61/64)^{100} = 0,0082$. El genoma de SARS-CoV-2, sin embargo, es rico en A y T ($p_A = 0,299$, $p_T = 0,321$, $p_G = 0,196$), lo que hace las paradas más frecuentes: la ecuación (0.13) da $\pi = 0,0664$, es decir, unas $9968 \times 0,0664 \approx 662$ paradas esperadas por marco, y un ORF aleatorio medio de solo 14 codones. La probabilidad de que un tramo aleatorio supere 100 codones cae a $(1 - 0,0664)^{100} = 0,0010$, así que en todo un marco esperaríamos $662 \times 0,0010 \approx 0,7$ ORF así de largos por puro azar. Los más de 7000 codones de ORF1ab son inexplicables por azar.

La figura 0.7 confirma la predicción: los genes destacan como tramos largos sin paradas. Pero la hebra complementaria contiene algunos ORF de más de 100 codones que no codifican nada, porque el genoma real no está hecho de bases independientes.

0.2.4 Archivos y registros: SeqRecord y SeqIO

Una secuencia aislada rara vez basta; necesitamos su identificador, su descripción y sus anotaciones. Biopython agrupa todo ello en un objeto SeqRecord, y el módulo SeqIO traduce entre registros y archivos en decenas de formatos. Los dos formatos que usaremos desde el primer día son FASTA, mínimo (una línea de encabezado que empieza con > seguida de la secuencia), y GenBank, rico en metadatos. Estudiaremos los formatos con detalle en el ??; aquí basta con saber que el encabezado de un GenBank se convierte en los atributos .id, .description y .annotations; su tabla de características, en una lista de objetos SeqFeature (con tipo, localización y calificadores), y su sección ORIGIN, en el atributo .seq.

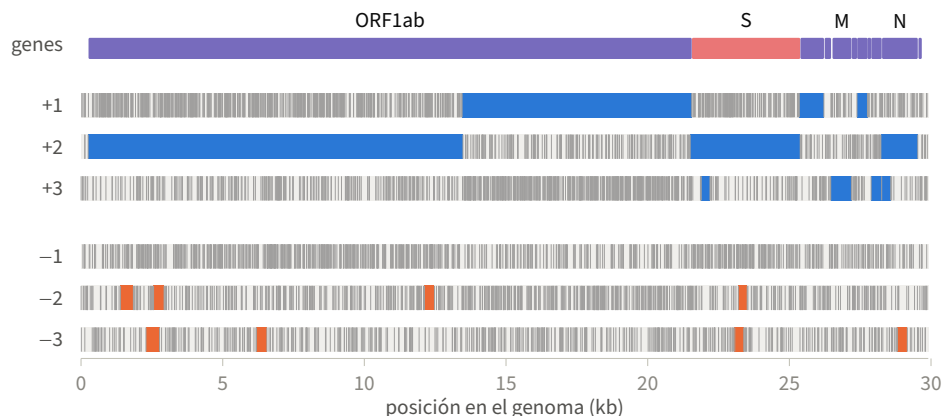


Figura 0.7. Los seis marcos de lectura del genoma de SARS-CoV-2. Cada raya gris es un codón de parada; los bloques de color son ORF de al menos 100 codones (azul, hebra directa; naranja, hebra complementaria). Arriba, los genes anotados en el GenBank. El marco +2 tiene solo 255 paradas, frente a las ~660 que predice la ecuación (0.13), porque contiene los más de 13 000 nucleótidos abiertos de ORF1a. En la hebra complementaria aparecen algunos ORF largos que no codifican nada, más de los que predice el modelo independiente: la longitud es una pista, no una prueba.

SeqIO ofrece dos funciones de lectura con una diferencia importante. `SeqIO.read` espera un archivo con *exactamente* un registro y falla si hay más, lo que protege contra errores silenciosos. `SeqIO.parse` devuelve un *iterador*: un objeto que produce los registros uno por uno, a medida que se le piden, sin cargar el archivo completo en memoria. Si un archivo contiene N registros de tamaño medio ℓ , leerlo todo en una lista necesita memoria $\Theta(N\ell)$, mientras que recorrerlo con el iterador necesita solo $\Theta(\ell)$, la del registro actual. Con un archivo FASTQ de 50 millones de lecturas, la diferencia es la que hay entre un análisis que funciona en Colab y otro que agota la memoria de la máquina.

0.2.5 Bases de datos programables: Entrez y el genoma de SARS-CoV-2

El Centro Nacional de Información Biotecnológica de Estados Unidos (NCBI) mantiene decenas de bases de datos enlazadas, desde GenBank y RefSeq hasta PubMed, y las expone mediante una interfaz programática, las E-utilities, que el sistema Entrez unifica (Sayers et al., 2022). El módulo `Bio.Entrez` traduce esas llamadas a funciones de Python: `esearch` busca identificadores que satisfacen una

consulta, `efetch` descarga los registros y `elink` navega entre bases de datos. El NCBI exige identificarse con un correo y limita las solicitudes por segundo.

```
descargar_genoma.py

from Bio import Entrez, SeqIO

Entrez.email = "su.correo@universidad.edu"      # obligatorio
acc = "NC_045512.2"
with Entrez.efetch(db="nucleotide", id=acc, rettype="gbwithparts",
                  retmode="text") as handle:
    rec = SeqIO.read(handle, "genbank")
    cds = [f for f in rec.features if f.type == "CDS"]
    print(rec.description, len(rec), "nt,", len(cds), "CDS")
```

El registro que descargamos es la secuencia de referencia del aislado Wuhan-Hu-1. Wu et al. (2020) lo obtuvieron por secuenciación metagenómica de ARN a partir del lavado broncoalveolar de un paciente hospitalizado en Wuhan en diciembre de 2019; el genoma completo mide 29 903 nucleótidos, y su análisis filogenético mostró que el nuevo virus estaba estrechamente emparentado (89,1 % de similitud nucleotídica) con un grupo de coronavirus similares al del SARS hallados en murciélagos. Es un excelente material de prácticas: cualquier resultado puede contrastarse con una literatura enorme.

Ejemplo 0.3. (Verificar una anotación y descubrir un salto de marco)

El GenBank anota 12 regiones codificantes (CDS) en 11 genes. Una prueba de consistencia natural es traducir cada CDS y comparar el resultado con la proteína que el propio archivo registra en el calificador `/translation`. Si extraemos cada región ingenuamente, desde su inicio hasta su final (con `seq[start:end]` y `translate`), 11 de las 12 traducciones coinciden. La excepción es la poliproteína ORF1ab, de 7096 aminoácidos: la traducción ingenua se detiene mucho antes. Si en cambio usamos `feature.extract(seq)`, que respeta la localización compuesta `join(266..13468,13468..21555)`, la traducción coincide residuo a residuo. La base 13 468 aparece en los dos intervalos: se lee dos veces. Es la huella de un *desplazamiento ribosómico del marco* en -1 : una estructura de ARN en forma de pseudonudo hace que, en una fracción de las traducciones, el ribosoma retroceda un nucleótido y continúe en otro marco. Así, un mismo ARN produce la poliproteína corta pp1a (sin salto, termina

en la parada del marco original) y la larga ppiab (con salto). La verificación computacional no solo detectó una “anomalía”: redescubrió un mecanismo biológico.

0.2.6 La línea de comandos y la filosofía Unix

Buena parte de las herramientas bioinformáticas más usadas (BLAST, BWA, samtools, SPAdes, IQ-TREE) son programas de línea de comandos escritos en C o C++. No tienen interfaz gráfica: se invocan escribiendo su nombre y sus opciones en una terminal, leen archivos o flujos de texto y escriben archivos o flujos de texto. Esta forma de trabajar se remonta al sistema operativo Unix. En su descripción original, Ritchie y Thompson (1974) presentaron, entre otras ideas, la *tubería (pipe)*: un canal que conecta la salida de un programa con la entrada del siguiente, de modo que programas pequeños, cada uno especializado en una tarea, se combinan para resolver problemas grandes.

Todo proceso de Unix tiene tres flujos estándar: la entrada (stdin), la salida (stdout) y el canal de errores (stderr), además de un *código de salida*, un entero que vale o si el programa terminó bien. Si vemos cada programa como una función que transforma texto en texto, una tubería es una composición de funciones:

$$p_1 | p_2 | \cdots | p_k \equiv f_k \circ \cdots \circ f_2 \circ f_1. \tag{0.15}$$

DONDE	
p_i	El i -ésimo programa de la tubería, con sus opciones.
$ $	Operador de tubería del intérprete de comandos: conecta stdout de la izquierda con stdin de la derecha.
f_i	Transformación de texto que realiza p_i ; la composición se lee de derecha a izquierda.

Los programas de una tubería se ejecutan *simultáneamente*: el segundo empieza a procesar las primeras líneas mientras el primero todavía está leyendo el archivo. Igual que el iterador de SeqIO.parse, una tubería procesa archivos mucho más grandes que la memoria disponible. La figura 0.8 muestra una tubería típica, que cuenta los tipos de anotación de un archivo GFF.

0.2.7 BLAST+ desde Python

BLAST es el ejemplo arquetípico de herramienta de línea de comandos: el conjunto BLAST+ reorganizó el código original en C++ y lo dividió en programas

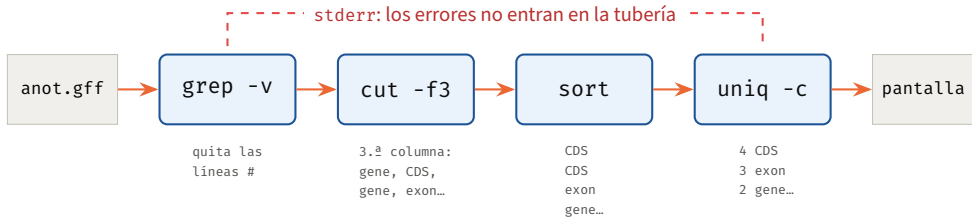


Figura 0.8. La tubería `grep -v '^#' anot.gff | cut -f3 | sort | uniq -c`. Cada programa lee el flujo de texto que le entrega el anterior (flechas naranjas, `stdout`→`stdin`), lo transforma y lo pasa al siguiente; debajo de cada uno, un fragmento de lo que produce. Los cuatro procesos corren a la vez y ninguno necesita cargar el archivo completo.

especializados, `makeblastdb` para indexar una base de datos y `blastn`, `blastp` y compañía para buscar en ella (Camacho et al., 2009). Estudiaremos su algoritmo y su estadística en el ???. Aquí nos interesa como instrumento: cómo se instala, cómo se invoca desde Python y cómo se lee lo que produce.

En Colab, BLAST+ se instala desde los repositorios del sistema (`apt-get install ncbi-blast+`); en un equipo propio conviene usar un gestor de entornos como `conda`, que veremos en la sección 0.3. Para llamarlo desde Python se usa el módulo estándar `subprocess`, pasando los argumentos como una lista y pidiendo `check=True` para que un código de salida distinto de cero se convierta en una excepción. El formato tabular `-outfmt 6` produce una fila por alineamiento, lista para `pandas`.

```

blast_subprocess.py

import subprocess
import pandas as pd

subprocess.run(["makeblastdb", "-in", "sarscov2.fasta",
               "-dbtype", "nucl", "-out", "sarscov2_db"], check=True)
cols = ("qseqid sseqid pident length mismatch gapopen "
        "qstart qend sstart send evalue bitscore").split()
subprocess.run(["blastn", "-query", "consulta.fasta",
               "-db", "sarscov2_db", "-out", "hits.tsv",
               "-outfmt", "6 " + " ".join(cols)], check=True)
hits = pd.read_csv("hits.tsv", sep="\t", names=cols)
print(hits.sort_values("bitscore", ascending=False).head())

```

¿Cuándo encuentra BLAST una secuencia mutada?

El cuaderno de la lección propone un experimento: tomar un fragmento de 300 nucleótidos del genoma, introducirle mutaciones al azar y comprobar si `blastn` lo encuentra. La respuesta depende de la estrategia de BLAST, que solo empieza a alinear donde encuentra una *semilla*: una palabra de w nucleótidos idéntica en la consulta y en la base de datos ($w = 28$ en el modo predeterminado de `blastn`, llamado megablast, y $w = 11$ con la opción `-task blastn`). Si cada posición conserva su base con probabilidad q (la identidad) de forma independiente, una palabra concreta sobrevive intacta con probabilidad q^w , y el número esperado de semillas en un fragmento de longitud ℓ es

$$E[\text{semillas}] = (\ell - w + 1) q^w. \quad (0.16)$$

DONDE

ℓ	Longitud del fragmento de consulta.
w	Longitud de palabra de la semilla (<code>-word_size</code>).
q	Identidad por posición entre la consulta y su origen, $q = 1 - \text{tasa de mutación}$.

Ejemplo 0.4. (Quince mutaciones en trescientas bases)

Con 15 mutaciones en 300 nucleótidos la identidad es $q = 0,95$. Con palabras de 11 bases, cada una sobrevive con probabilidad $0,95^{11} = 0,569$; hay $300 - 11 + 1 = 290$ palabras, así que esperamos unas $290 \times 0,569 \approx 165$ semillas. Con el modo predeterminado ($w = 28$) la supervivencia cae a $0,95^{28} = 0,238$, pero sobre 273 palabras aún quedan unas 65 semillas: BLAST encontrará el fragmento en ambos casos. El panorama cambia con $q = 0,80$ (60 mutaciones): con $w = 11$ esperamos todavía $290 \times 0,80^{11} \approx 25$ semillas, mientras que con $w = 28$ solo $273 \times 0,80^{28} \approx 0,5$. Es muy probable que megablast no encuentre nada. Para homólogos aún más lejanos, la búsqueda de ADN pierde sensibilidad incluso con palabras cortas y conviene comparar proteínas.

El ejemplo resume el compromiso: el tamaño de palabra regula el equilibrio entre velocidad y sensibilidad, y es uno de los parámetros que conviene registrar siempre junto con los resultados. Que las mutaciones se distribuyan al azar es, a su vez, un supuesto: en genes reales las mutaciones se concentran en las terceras posiciones de los codones, y eso favorece a las semillas “discontinuas” que ignoran esas posiciones, una idea que retomaremos en el ??.

0.3 Reproducibilidad: semillas, versiones, hashes y Git

 Colab · 0.3 · Reproducibilidad

 abrir notebook

La receta de la abuela

“Agregue un poco de harina, una pizca de sal y hornee hasta que esté listo”. La receta funciona para quien la escribió, que sabe cuánto es “un poco” y reconoce el punto exacto del horno, pero nadie más obtiene el mismo pastel. Un análisis bioinformático sin semillas aleatorias registradas, sin versiones de los programas y sin una huella de los datos de entrada es esa receta: funcionó una vez, en una computadora, un día concreto, y nadie, ni siquiera su autor seis meses después, puede garantizar que volverá a obtener lo mismo.

0.3.1 ¿Qué significa reproducir un análisis?

La palabra “reproducibilidad” se usa con sentidos distintos, y conviene separarlos. Hablaremos de *repetibilidad* cuando el mismo investigador, con los mismos datos, el mismo código y el mismo entorno, obtiene el mismo resultado; de *reproducibilidad computacional* cuando lo obtiene otra persona a partir de los datos y el código publicados; y de *replicabilidad* cuando un estudio independiente, con datos nuevos, llega a la misma conclusión. La reproducibilidad computacional es el requisito mínimo: sin ella, una discrepancia con otro estudio no puede atribuirse ni a la biología ni a un error.

El problema no es hipotético. En una encuesta de *Nature* a 1576 investigadores, más del 70 % declaró haber intentado sin éxito reproducir un experimento de otro grupo, y más de la mitad, uno propio (Baker, 2016). En bioinformática la evidencia es más directa. Ioannidis et al. (2009) encargaron a dos equipos independientes reproducir una tabla o figura de 18 análisis de expresión con micromatrices publicados en *Nature Genetics*: lograron reproducir dos en principio y seis de forma parcial o con discrepancias; los diez restantes no pudieron reproducirse. La causa principal fue la falta de disponibilidad de los datos, y las discrepancias se debieron sobre todo a descripciones incompletas del procesamiento.

Un análisis computacional puede dejar de ser reproducible por cuatro vías: porque depende de *números aleatorios* no registrados, porque sus *datos* cambiaron sin que

nadie lo notara, porque su *código* se modificó después de producir los resultados, o porque su *entorno* (versiones de bibliotecas, del sistema operativo) ya no es el mismo. El resto del capítulo presenta un instrumento para cada vía.

0.3.2 El azar determinista: generadores pseudoaleatorios

Simulaciones, remuestreos *bootstrap*, inicializaciones de algoritmos de agrupamiento, divisiones de datos para entrenar un modelo: la bioinformática consume cantidades enormes de números aleatorios. Pero una computadora es una máquina determinista y no puede lanzar una moneda. Lo que hace es calcular una sucesión que *parece* aleatoria.

Definición 0.5. (Generador pseudoaleatorio)

Un generador de números pseudoaleatorios es una tupla $(\mathcal{S}, f, g, s_0)$ formada por un conjunto finito de estados $\mathcal{S}$, una función de transición $f : \mathcal{S} \rightarrow \mathcal{S}$, una función de salida $g : \mathcal{S} \rightarrow [0, 1)$ y un estado inicial s_0 , la *semilla*. El generador produce $u_n = g(s_n)$ con $s_{n+1} = f(s_n)$. Como $\mathcal{S}$ es finito, la sucesión es tarde o temprano periódica; el menor P tal que $s_{n+P} = s_n$ para todo n grande es su *período*.

La definición contiene la clave de la reproducibilidad: toda la sucesión queda determinada por la semilla. Es como una baraja mezclada de antemano: la semilla dice en qué carta empezar.

El generador más antiguo y más estudiado es el *congruencial lineal* (LCG), cuyo estado es un único entero:

$$x_{n+1} = (a x_n + c) \bmod m, \quad u_n = \frac{x_n}{m}. \quad (0.17)$$

DONDE

x_n	Estado del generador, un entero entre 0 y $m - 1$; x_0 es la semilla.
a, c, m	Multiplicador, incremento y módulo: constantes que definen el generador.
u_n	Número pseudoaleatorio en $[0, 1)$ que se entrega al usuario.

El período de un LCG no puede superar m , y alcanzar ese máximo exige elegir bien las constantes. El resultado clásico, que Knuth (1997) demuestra con detalle, es el siguiente.

Teorema 0.2. (Período completo de un LCG (Hull–Dobell))

Si $c \neq 0$, el generador de la ecuación (0.17) tiene período m para toda semilla s y solo si: (i) c y m son primos entre sí; (ii) $a - 1$ es divisible por todos los factores primos de m ; y (iii) $a - 1$ es divisible por 4 si m lo es.

Ejemplo 0.5. (Un generador de juguete con período completo)

Tome $m = 16$, $a = 5$, $c = 3$. Las condiciones se cumplen: $\text{mcd}(3, 16) = 1$; el único primo de 16 es 2, que divide a $a - 1 = 4$; y como $4 \mid 16$, también $4 \mid (a - 1)$. Con semilla $x_0 = 7$, la sucesión es

$$\begin{aligned} 7 \rightarrow 6 \rightarrow 1 \rightarrow 8 \rightarrow 11 \rightarrow 10 \rightarrow 5 \rightarrow 12 \rightarrow 15 \\ \rightarrow 14 \rightarrow 9 \rightarrow 0 \rightarrow 3 \rightarrow 2 \rightarrow 13 \rightarrow 4 \rightarrow 7 \rightarrow \dots \end{aligned}$$

que recorre los 16 estados antes de repetirse. Con $a = 3$, en cambio, la condición (ii) se cumple pero la (iii) no, y el período se acorta. Observe también que los bits bajos son pésimos: $x_n \bmod 2$ alterna 1, 0, 1, 0..., un defecto general de los LCG con módulo potencia de 2.

0.3.3 Cuando el azar tiene estructura: RANDU

Período largo no significa buena calidad. El caso más famoso es RANDU, el generador de la biblioteca científica de IBM en los años sesenta, con $a = 65539$, $c = 0$ y $m = 2^{31}$, usado durante años en simulaciones publicadas. Su multiplicador tiene una forma especial, $a = 2^{16} + 3$, y eso basta para arruinarlo. Elevando al cuadrado,

$$a^2 = 2^{32} + 6 \cdot 2^{16} + 9 \equiv 6 \cdot 2^{16} + 9 = 6(2^{16} + 3) - 9 = 6a - 9 \pmod{2^{31}}, \quad (0.18)$$

porque 2^{32} es múltiplo de 2^{31} . Multiplicando por x_n y usando $x_{n+1} \equiv ax_n$ y $x_{n+2} \equiv a^2x_n$, obtenemos una relación lineal entre tres valores consecutivos:

$$x_{n+2} \equiv 6x_{n+1} - 9x_n \pmod{2^{31}} \implies 9u_n - 6u_{n+1} + u_{n+2} = k \in \mathbb{Z}. \quad (0.19)$$

DONDE

u_n	Salida normalizada $x_n/2^{31} \in [0, 1)$.
k	Entero; como $9u_n - 6u_{n+1} + u_{n+2}$ está entre -6 y 10 , solo puede tomar 15 valores $(-5, \dots, 9)$.

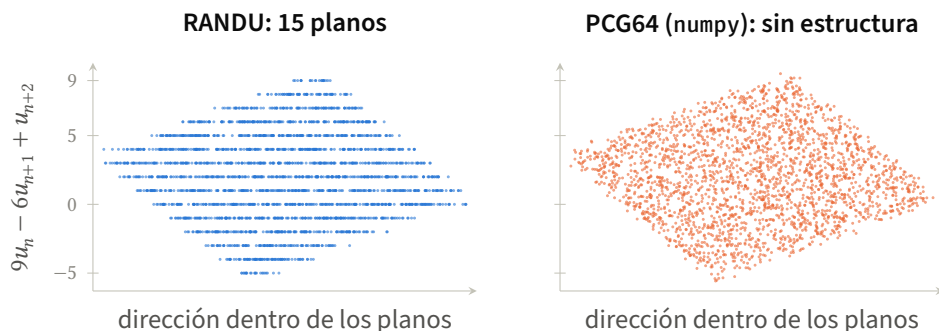


Figura 0.9. 2500 tripletes consecutivos (u_n, u_{n+1}, u_{n+2}) proyectados sobre dos direcciones: el eje vertical es la combinación $9u_n - 6u_{n+1} + u_{n+2}$ de la ecuación (0.19) y el horizontal, una dirección perpendicular a ella. Los tripletes de RANDU (izquierda) solo toman 15 valores enteros en el eje vertical: son 15 planos vistos de canto. Los de PCG64, el generador predeterminado de NumPy (derecha), llenan el espacio de forma homogénea.

La ecuación (0.19) dice que todos los tripletes (u_n, u_{n+1}, u_{n+2}) caen sobre 15 planos paralelos, con vector normal $(9, -6, 1)$ (figura 0.9). Una simulación tridimensional con RANDU explora solo 15 láminas del cubo y deja vacío todo lo demás. Marsaglia (1968) demostró que el defecto es inherente a todos los generadores congruenciales multiplicativos, aunque en grado muy distinto: los puntos de d valores consecutivos caen siempre sobre una familia de hiperplanos paralelos, cuyo número no puede superar $(d! m)^{1/d}$. Para $m = 2^{31}$ y $d = 3$ esa cota es de unos 2344 planos; RANDU se queda con 15.

0.3.4 Generadores modernos: Mersenne Twister y PCG

Los generadores actuales se diseñan con pruebas teóricas de uniformidad en muchas dimensiones y se someten a baterías de pruebas estadísticas. El Mersenne Twister (MT19937) de Matsumoto y Nishimura (1998), el generador del módulo random de Python y durante años el de NumPy, tiene un período de $2^{19937} - 1$ y está equidistribuido en 623 dimensiones: una garantía que RANDU no cumple ni en tres. Pero es lineal sobre el cuerpo de dos elementos: unas cuantas salidas consecutivas bastan para reconstruir su estado, así que sirve para simular, nunca para criptografía.

Desde la versión 1.17, NumPy usa por defecto PCG64, un generador de la familia de congruenciales permutados propuesta por Melissa O'Neill: un LCG de 128 bits cuyo estado no se entrega directamente, sino a través de una permutación que mezcla

sus bits (se combinan sus dos mitades con un o exclusivo y el resultado se rota un número de posiciones que dictan sus bits altos), lo que elimina las regularidades de los bits bajos que vimos en el ejemplo 0.5. Tiene período 2^{128} , un estado pequeño y puede “saltar” hacia adelante en la sucesión de forma eficiente, lo que permite crear flujos independientes para cálculos en paralelo (NumPy Developers, 2024).

La forma moderna de usarlo es crear un generador propio con una semilla explícita y pasarlo a las funciones que lo necesitan, en lugar de fijar el estado global con `np.random.seed`, que cualquier biblioteca importada puede alterar sin aviso.

semillas.py

```
import numpy as np

rng = np.random.default_rng(20240517)          # semilla explícita
reads = rng.choice(list("ACGT"), size=(3, 12))
print(["".join(r) for r in reads])              # idéntico en cada ejecución

# flujos independientes para 4 procesos en paralelo
children = np.random.SeedSequence(20240517).spawn(4)
streams = [np.random.default_rng(s) for s in children]
```

La misma semilla solo reproduce la misma sucesión con el mismo generador y la misma versión del código; por eso se registra junto con las versiones del software.

0.3.5 Monte Carlo y su error

Una simulación Monte Carlo estima una cantidad (una probabilidad, una media) promediando muchas repeticiones aleatorias. Si $X_1, \dots, X_N$ son repeticiones independientes de una variable con media μ y desviación estándar σ , el estimador es la media muestral $\hat{\mu}_N$, y su precisión obedece a un resultado fundamental.

Teorema 0.3. (Error estándar de Monte Carlo)

El estimador $\hat{\mu}_N = \frac{1}{N} \sum_{i=1}^N X_i$ es insesgado, $E[\hat{\mu}_N] = \mu$, y su error estándar es

$$EE(\hat{\mu}_N) = \sqrt{\text{Var}(\hat{\mu}_N)} = \frac{\sigma}{\sqrt{N}}. \quad (0.20)$$

Por el teorema central del límite, $\hat{\mu}_N \pm 2\sigma/\sqrt{N}$ contiene a μ con probabilidad cercana al 95 %.

DONDE

X_i	Resultado de la i -ésima repetición (por ejemplo, 1 si una lectura simulada no tiene errores y 0 si los tiene).
μ, σ	Media y desviación estándar verdaderas de X_i para una variable de Bernoulli, $\sigma = \sqrt{p(1-p)}$.
N	Número de repeticiones de la simulación.

La demostración es directa: por independencia, $\text{Var}(\sum X_i) = N\sigma^2$, y dividir entre N divide la varianza entre N^2 . La consecuencia práctica es la ley de la raíz cuadrada: *para ganar un decimal de precisión hay que multiplicar el trabajo por cien*. Invirtiendo la ecuación (0.20), el número de repeticiones necesario para un error estándar ϵ es $N = \sigma^2/\epsilon^2$.

Ejemplo 0.6. (Lecturas sin errores)

Un secuenciador comete un error en cada base con probabilidad $e = 0,01$, de forma independiente. ¿Qué fracción de las lecturas de 150 bases no tiene ningún error? La respuesta exacta es $p = (1 - e)^{150} = 0,99^{150} = 0,2215$; se parece a $e^{-1,5} = 0,2231$, la aproximación de Poisson. Si no conociéramos la fórmula, podríamos simular $N = 10\,000$ lecturas y contar las perfectas. El error estándar sería $\sqrt{0,2215 \times 0,7785 / 10\,000} = 0,0042$, y las cuatro simulaciones de la figura 0.10 terminan en 0,2195, 0,2240, 0,2204 y 0,2223 tras 10^5 lecturas: todas dentro de la banda teórica. Para obtener la tercera cifra decimal con seguridad ($\epsilon = 0,0005$) harían falta unas $0,17/0,0005^2 \approx 690\,000$ lecturas simuladas.

0.3.6 Huellas digitales: funciones hash

Los datos también cambian sin avisar. Un archivo se trunca durante una descarga, una base de datos publica una nueva versión con el mismo nombre de archivo, un colaborador “limpia” una tabla y la guarda encima de la original. Para detectar cualquier cambio, por pequeño que sea, se usa la huella digital del archivo, el resultado de una función *hash* criptográfica.

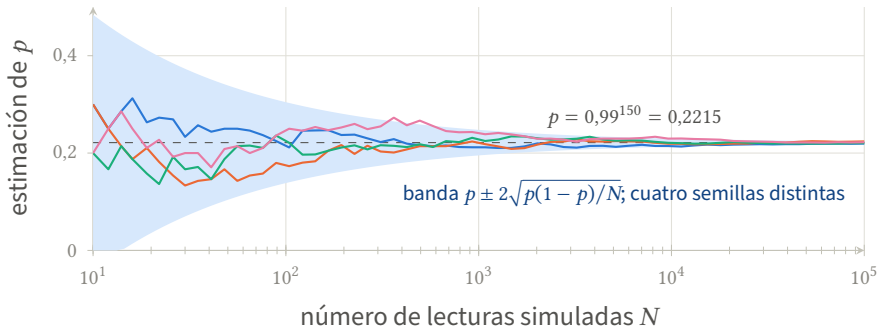


Figura 0.10. Estimación Monte Carlo de la fracción de lecturas de 150 bases sin errores de secuenciación ($e = 0,01$), con cuatro semillas distintas (líneas de color). Al principio las estimaciones difieren mucho; a medida que crece N convergen hacia el valor exacto (línea discontinua) y se mantienen dentro de la banda de ± 2 errores estándar, cuyo ancho se reduce como $1/\sqrt{N}$: en escala logarítmica, cada década de N estrecha la banda en un factor $\sqrt{10} \approx 3,2$.

Definición 0.6. (Función *hash* criptográfica)

Una función $h : \{0, 1\}^* \rightarrow \{0, 1\}^n$ que asigna a cualquier mensaje, de cualquier longitud, un resumen de n bits, y que satisface tres propiedades: (1) *resistencia a la preimagen*: dado un resumen y , es inviable encontrar un mensaje x con $h(x) = y$; (2) *resistencia a la segunda preimagen*: dado x , es inviable encontrar $x' \neq x$ con $h(x') = h(x)$; (3) *resistencia a colisiones*: es inviable encontrar cualquier par $x \neq x'$ con $h(x) = h(x')$.

SHA-256, definida en el estándar federal estadounidense de *hash* seguro (National Institute of Standards and Technology, 2015), produce resúmenes de $n = 256$ bits, que se escriben como 64 dígitos hexadecimales. Procesa el mensaje en bloques de 512 bits, cada uno mediante 64 rondas de sumas modulares, rotaciones y funciones lógicas, diseñadas para que cada bit de la salida dependa de todos los bits de la entrada. El resultado es el *efecto avalancha*: cambiar un solo bit de la entrada cambia, en promedio, la mitad de los bits de la salida, y el resumen modificado no guarda ninguna relación visible con el original.

Ejemplo 0.7. (Una base de diferencia)

Las secuencias **GATTACA** y **GATTACC** difieren en una sola base. Sus resúmenes SHA-256 son

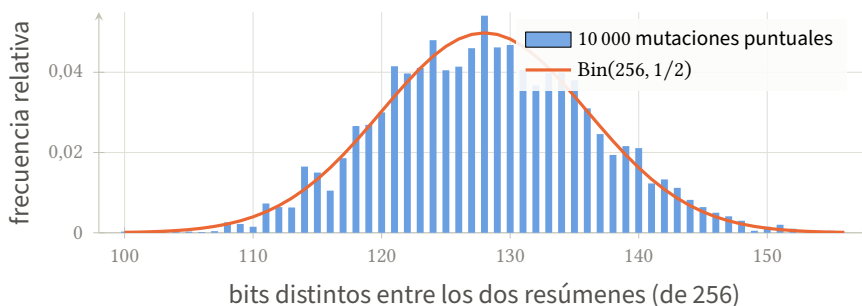


Figura 0.11. Efecto avalancha. Se tomó una secuencia aleatoria de 1000 bases y se le aplicaron 10 000 mutaciones puntuales independientes; para cada una se contaron los bits que difieren entre el SHA-256 de la secuencia original y el de la mutada (media observada 128,2; desviación estándar 8,2). La distribución coincide con la de 256 monedas justas (naranja): un cambio mínimo en los datos produce una huella completamente distinta, y nada en la huella delata dónde estuvo el cambio.

```
d74f6c423e80cbf69d76149048e458a10c96f927c896ea9ff4f44616b643eb22
60bcb9d965bb9c40959b91781d457c8801eca61d70fee26377d612d205eecb7
```

y difieren en 129 de sus 256 bits. Si la salida fuera una sucesión de bits aleatorios e independientes, el número de bits distintos seguiría una binomial $\text{Bin}(256, \frac{1}{2})$, con media 128 y desviación estándar 8. La figura 0.11 muestra que eso es exactamente lo que ocurre.

¿Pueden dos archivos distintos tener la misma huella?

Sí, necesariamente: hay infinitos mensajes y solo 2^{256} resúmenes. La pregunta relevante es con qué probabilidad ocurrirá *por accidente*. Es el problema del cumpleaños: si k archivos reciben resúmenes que se comportan como uniformes e independientes sobre $M = 2^n$ valores, la probabilidad de que no haya ninguna coincidencia es

$$\mathbb{P}(\text{sin colisión}) = \prod_{i=1}^{k-1} \left(1 - \frac{i}{M}\right) \approx \exp\left(-\frac{k(k-1)}{2M}\right), \quad (0.21)$$

usando $1 - x \approx e^{-x}$ para x pequeño y $\sum_{i=1}^{k-1} i = k(k-1)/2$.

DONDE

k	Número de archivos a los que se calcula la huella.
$M = 2^n$	Número de resúmenes posibles; $n = 256$ para SHA-256.

La probabilidad de colisión se vuelve apreciable solo cuando k se acerca a $\sqrt{M} = 2^{n/2}$. Para SHA-256 eso significa $2^{128} \approx 3 \times 10^{38}$ archivos. Con un billón (10^{12}) de archivos, la probabilidad de alguna colisión accidental es del orden de $10^{24}/2^{257} \approx 4 \times 10^{-54}$. Para efectos prácticos, dos archivos con el mismo SHA-256 son el mismo archivo.

Verificar la integridad de los datos

```
# calcular y guardar las huellas de los datos crudos
sha256sum datos/*.fastq.gz > datos/SHA256SUMS
# meses después, u otra persona: ¿son los mismos archivos?
sha256sum -c datos/SHA256SUMS
# lecturas_R1.fastq.gz: OK
# lecturas_R2.fastq.gz: FAILED    <- el archivo cambió
```

0.3.7 Git: el historial como grafo

La tercera vía de pérdida de reproducibilidad es el código. Carpetas llenas de `analisis_final.py`, `analisis_final_v2.py` y `analisis_final_DEFINITIVO.py` son la versión computacional del cuaderno de laboratorio con páginas arrancadas. Un sistema de control de versiones registra cada estado del proyecto, quién lo produjo, cuándo y por qué, y permite volver a cualquiera de ellos. Git, el sistema dominante, tiene un modelo interno sorprendentemente sencillo, y entenderlo evita la mayoría de los sustos.

Definición 0.7. (Historial de Git)

Un *commit* es una instantánea completa de los archivos del proyecto, acompañada de metadatos (autor, fecha, mensaje) y de la referencia a uno o más commits *padres*. Cada commit se identifica por el *hash* de todo su contenido, incluidos los identificadores de sus padres. El historial es un *grafo acíclico dirigido* (DAG) $G = (V, E)$: los vértices V son commits y hay una arista $(c \rightarrow p) \in E$ si p es padre de c .

incluya el identificador del commit en cada figura o tabla generada, y guarde los datos grandes fuera de Git, referenciados por su huella.

0.3.8 Entornos, versiones y contenedores

La cuarta vía es el entorno. El mismo código puede producir resultados distintos con otra versión de NumPy, de Biopython o de BLAST, y un análisis que hoy funciona puede fallar mañana porque Colab actualizó una biblioteca. La defensa tiene varias capas, de menor a mayor aislamiento, y un análisis es tan reproducible como la capa más débil que haya quedado sin registrar.

La capa mínima es *registrar*: guardar junto con los resultados la lista exacta de versiones (`pip freeze`, `Bio.__version__`, `blastn -version`), por ejemplo en la primera celda de cada cuaderno. La siguiente es *fixar*: declarar esas versiones en un archivo (`requirements.txt` o `environment.yml`) a partir del cual cualquiera pueda recrear el entorno. Con versionado semántico (MAYOR.MENOR.PARCHE), fijar la versión mayor (`numpy>=1.26,<2`) evita rupturas, y fijar la exacta (`numpy==1.26.4`) garantiza además los mismos resultados numéricos.

La capa más robusta es el *contenedor*, una imagen que empaqueta el programa con todas las bibliotecas del sistema de las que depende y que se ejecuta igual en cualquier computadora con el motor adecuado. Grüning et al. (2018) describen cómo combinar estas capas en la práctica: gestores de paquetes como conda y su canal Bioconda para instalar software bioinformático con versiones exactas, contenedores generados automáticamente a partir de esos paquetes, y sistemas de gestión de flujos de trabajo que registran cada paso.

0.3.9 Reglas y principios: de Sandve a FAIR

Las herramientas de esta sección responden a un conjunto de principios que la comunidad ha ido formulando. Sandve et al. (2013) los condensaron en diez reglas simples para la investigación computacional reproducible, que la tabla 0.1 relaciona con los instrumentos de este capítulo. En paralelo, Wilson et al. (2014) reunieron las *mejores prácticas* de la computación científica (escribir programas para personas, automatizar lo repetitivo, hacer cambios pequeños y versionarlos, planear para los errores), y Wilson et al. (2017) propusieron un conjunto “suficientemente bueno” al alcance de cualquier grupo: guardar los datos crudos sin modificar, dar a cada proyecto una estructura predecible, versionar el código y documentar cómo se ejecuta.

Los principios FAIR amplían la mirada de los análisis a los *datos*. Wilkinson et al. (2016) propusieron que los datos científicos sean *localizables* (*Findable*: con

Tabla 0.1. Las diez reglas de Sandve et al. (2013) para la investigación computacional reproducible (traducción libre) y el instrumento del capítulo que ayuda a cumplirlas.

Regla	Instrumento
1 Registre cómo se produjo cada resultado.	cuaderno, commit
2 Evite los pasos manuales de manipulación de datos.	código, pandas
3 Archive las versiones exactas de los programas externos.	<code>pip freeze</code> , conda
4 Ponga bajo control de versiones todos sus <i>scripts</i> .	Git
5 Registre los resultados intermedios, en formatos estándar si es posible.	CSV, FASTA
6 En los análisis con aleatoriedad, anote las semillas.	<code>default_rng</code>
7 Guarde siempre los datos crudos detrás de cada gráfica.	<code>.dat</code> , <code>.csv</code>
8 Genere salidas jerárquicas, que permitan inspeccionar niveles de detalle.	tablas y figuras
9 Conecte las afirmaciones del texto con los resultados que las sustentan.	cuaderno
10 Dé acceso público a los <i>scripts</i> , las ejecuciones y los resultados.	GitHub

identificadores persistentes y metadatos ricos, registrados en recursos donde se puedan buscar), *accesibles* (*Accessible*: recuperables mediante protocolos abiertos y estandarizados, con metadatos que persisten aunque los datos dejen de estar disponibles), *interoperables* (*Interoperable*: descritos con vocabularios y formatos compartidos) y *reutilizables* (*Reusable*: con licencia clara, procedencia documentada y conformes a los estándares de su comunidad). Una idea central del artículo es que los datos deben ser legibles no solo por personas sino también por máquinas. Un número de acceso como NC_045512.2, con su versión explícita, es un buen ejemplo: el sufijo de versión .2 garantiza que cualquier cambio futuro recibirá un identificador nuevo.

Semilla, huella, commit y entorno:
cuatro registros baratos que convierten una receta en un protocolo.

En resumen

- Un *notebook* separa el documento del núcleo que lo ejecuta; el estado del núcleo depende del orden de ejecución, y solo “reiniciar y ejecutar todo” garantiza que el documento y sus resultados coinciden.
- Una secuencia es un arreglo de símbolos; su costo en memoria depende de la codificación ($nb/8$ bytes) y el costo de un algoritmo se resume con la notación O . La vectorización con NumPy no cambia la complejidad, pero reduce la constante en dos órdenes de magnitud.
- El punto flotante tiene error relativo $\approx 10^{-16}$ y se desborda por abajo al multiplicar probabilidades: se trabaja con logaritmos y log-suma-exp.
- Los datos ordenados y las figuras diseñadas según la percepción forman parte del método, no de la estética.
- Biopython representa secuencias (Seq), registros (SeqRecord) y archivos (SeqIO); Entrez descarga datos del NCBI; las herramientas de línea de comandos se combinan con tuberías y se llaman desde Python con subprocess.
- Un generador pseudoaleatorio es una máquina determinista cuya salida fija la semilla; RANDU muestra que un mal generador introduce estructura (15 planos), y los modernos (Mersenne Twister, PCG64) se diseñan para evitarla. El error Monte Carlo decrece como $\sigma/\sqrt{N}$.
- SHA-256 da a cada archivo una huella de 256 bits con efecto avalancha y colisiones accidentales prácticamente imposibles; Git encadena huellas en un grafo acíclico dirigido de commits.

Ejercicios

Dificultad: ●●● básica ●●● intermedia ●●● avanzada

- 0.1** ●●● Un gen está anotado en las posiciones 21 563..25 384 de un genoma (numeración de GenBank). Escriba el corte de Python que lo extrae y calcule su longitud en nucleótidos y en codones. ¿Qué error se comete si se escribe `seq[21563:25384]`?
- 0.2** ●●● Calcule cuánta memoria ocupa el genoma de *Escherichia coli* ($4,6 \times 10^6$ pb) en cada una de las representaciones descritas tras la ecuación (0.2). ¿Cuál elegiría para calcular un *dot plot* contra otro genoma del mismo tamaño?

- 0.3** ●●● Implemente `logsumexp(a)` a partir de la ecuación (0.7) y compárela con `scipy.special.logsumexp` para $a = [-1000, -1001, -1002]$ y para $a = [1000, 1001]$. ¿Por qué falla la evaluación directa en cada caso?
- 0.4** ●●● Repita el ejemplo 0.1 con ventanas de 100, 500 y 2000 bases. ¿Cómo cambia la fracción de ventanas fuera de la banda binomial? Interprete el resultado en términos de ruido de muestreo y de heterogeneidad biológica.
- 0.5** ●●● Demuestre la aproximación de la ecuación (0.21) y úsela para calcular cuántos archivos harían falta para tener un 50 % de probabilidad de colisión con MD5 (128 bits) y con SHA-256. Compare con el número estimado de archivos almacenados en el mundo (del orden de 10^{15}).
- 0.6** ●●● Escriba una función que reciba una semilla, simule N lecturas de 150 bases con una tasa de error por base e y devuelva la estimación de la fracción de lecturas sin errores con su error estándar. Determine empíricamente el N necesario para que el intervalo de ± 2 errores estándar tenga un ancho menor que 0,001 y compárelo con la predicción de la ecuación (0.20).
- 0.7** ●●● Tome un análisis propio (o el cuaderno de la sección 0.2) y evalúelo frente a las diez reglas de la tabla 0.1. Para cada regla incumplida, proponga el cambio concreto (un archivo, una línea de código, un comando) que la haría cumplir.

Referencias del capítulo

- Baker, M. (2016). 1,500 scientists lift the lid on reproducibility. *Nature*, 533(7604), 452-454. <https://doi.org/10.1038/533452a>
- Buckheit, J. B., & Donoho, D. L. (1995). WaveLab and Reproducible Research. En *Wavelets and Statistics* (pp. 55-81). Springer New York. https://doi.org/10.1007/978-1-4612-2544-7_5
- Camacho, C., Coulouris, G., Avagyan, V., Ma, N., Papadopoulos, J., Bealer, K., & Madden, T. L. (2009). BLAST+: architecture and applications. *BMC Bioinformatics*, 10(1), Artículo 421. <https://doi.org/10.1186/1471-2105-10-421>
- Cleveland, W. S., & McGill, R. (1984). Graphical Perception: Theory, Experimentation, and Application to the Development of Graphical Methods. *Journal of the American Statistical Association*, 79(387), 531-554. <https://doi.org/10.1080/01621459.1984.10478080>

- Cock, P. J. A., Antao, T., Chang, J. T., Chapman, B. A., Cox, C. J., Dalke, A., Friedberg, I., Hamelryck, T., Kauff, F., Wilczynski, B., & de Hoon, M. J. L. (2009). Biopython: freely available Python tools for computational molecular biology and bioinformatics. *Bioinformatics*, 25(11), 1422-1423. <https://doi.org/10.1093/bioinformatics/btp163>
- Cramer, F., Shephard, G. E., & Heron, P. J. (2020). The misuse of colour in science communication. *Nature Communications*, 11(1), Artículo 5444. <https://doi.org/10.1038/s41467-020-19160-7>
- Goldberg, D. (1991). What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys*, 23(1), 5-48. <https://doi.org/10.1145/103162.103163>
- Grüning, B., Chilton, J., Köster, J., Dale, R., Soranzo, N., van den Beek, M., Goecks, J., Backofen, R., Nekrutenko, A., & Taylor, J. (2018). Practical Computational Reproducibility in the Life Sciences. *Cell Systems*, 6(6), 631-635. <https://doi.org/10.1016/j.cels.2018.03.014>
- Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., ... Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825), 357-362. <https://doi.org/10.1038/s41586-020-2649-2>
- Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering*, 9(3), 90-95. <https://doi.org/10.1109/MCSE.2007.55>
- Ioannidis, J. P. A., Allison, D. B., Ball, C. A., Coulibaly, I., Cui, X., Culhane, A. C., Falchi, M., Furlanello, C., Game, L., Jurman, G., Mangion, J., Mehta, T., Nitzberg, M., Page, G. P., Petretto, E., & van Noort, V. (2009). Repeatability of published microarray gene expression analyses. *Nature Genetics*, 41(2), 149-155. <https://doi.org/10.1038/ng.295>
- Kluyver, T., Ragan-Kelley, B., Pérez, F., Granger, B., Bussonnier, M., Frederic, J., Kelley, K., Hamrick, J., Grout, J., Corlay, S., Ivanov, P., Avila, D., Abdalla, S., Willing, C., & Jupyter Development Team. (2016). Jupyter Notebooks – a publishing format for reproducible computational workflows. En *Positioning and Power in Academic Publishing: Players, Agents and Agendas* (pp. 87-90). IOS Press. <https://doi.org/10.3233/978-1-61499-649-1-87>
- Knuth, D. E. (1997). *The Art of Computer Programming* (3.^a ed., Vol. 2). Addison-Wesley.
- Marsaglia, G. (1968). Random numbers fall mainly in the planes. *Proceedings of the National Academy of Sciences*, 61(1), 25-28. <https://doi.org/10.1073/pnas.61.1.25>
- Matsumoto, M., & Nishimura, T. (1998). Mersenne twister: a 623-dimensionally equidistributed uniform pseudo-random number generator. *ACM Transactions on Modeling and Computer Simulation*, 8(1), 3-30. <https://doi.org/10.1145/272991.272995>
- McKinney, W. (2010). Data Structures for Statistical Computing in Python. *Proceedings of the 9th Python in Science Conference*, 56-61. <https://doi.org/10.25080/Majora-92bf1922-00a>
- National Institute of Standards and Technology. (2015). *Secure Hash Standard* (Federal Information Processing Standards Publication N.º 180-4). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.FIPS.180-4>

- NumPy Developers. (2024). *Permuted Congruential Generator (64-bit, PCG64)*. NumPy documentation. https://numpy.org/doc/stable/reference/random/bit_generators/pcg64.html
- Perez-Riverol, Y., Gatto, L., Wang, R., Sachsenberg, T., Uszkoreit, J., Leprevost, F. d. V., Fufezan, C., Ternent, T., Eglen, S. J., Katz, D. S., Pollard, T. J., Kononov, A., Flight, R. M., Blin, K., & Vizcaino, J. A. (2016). Ten Simple Rules for Taking Advantage of Git and GitHub. *PLOS Computational Biology*, 12(7), Artículo e1004947. <https://doi.org/10.1371/journal.pcbi.1004947>
- Ritchie, D. M., & Thompson, K. (1974). The UNIX time-sharing system. *Communications of the ACM*, 17(7), 365-375. <https://doi.org/10.1145/361011.361061>
- Sandve, G. K., Nekrutenko, A., Taylor, J., & Hovig, E. (2013). Ten Simple Rules for Reproducible Computational Research. *PLoS Computational Biology*, 9(10), Artículo e1003285. <https://doi.org/10.1371/journal.pcbi.1003285>
- Sayers, E. W., Bolton, E. E., Brister, J. R., Canese, K., Chan, J., Comeau, D. C., Connor, R., Funk, K., Kelly, C., Kim, S., Madej, T., Marchler-Bauer, A., Lanczycki, C., Lathrop, S., Lu, Z., Thibaud-Nissen, F., Murphy, T., Phan, L., Skripchenko, Y., ... Sherry, S. T. (2022). Database resources of the National Center for Biotechnology Information. *Nucleic Acids Research*, 50(D1), D20-D26. <https://doi.org/10.1093/nar/gkab1112>
- Tufte, E. R. (2001). *The Visual Display of Quantitative Information* (2.^a ed.). Graphics Press.
- Wickham, H. (2014). Tidy Data. *Journal of Statistical Software*, 59(10), 1-23. <https://doi.org/10.18637/jss.v059.i10>
- Wilkinson, M. D., Dumontier, M., Aalbersberg, I. J., Appleton, G., Axton, M., Baak, A., Blomberg, N., Boiten, J.-W., da Silva Santos, L. B., Bourne, P. E., et al. (2016). The FAIR Guiding Principles for scientific data management and stewardship. *Scientific Data*, 3, Artículo 160018. <https://doi.org/10.1038/sdata.2016.18>
- Wilson, G., Aruliah, D. A., Brown, C. T., Chue Hong, N. P., Davis, M., Guy, R. T., Haddock, S. H. D., Huff, K. D., Mitchell, I. M., Plumbley, M. D., Waugh, B., White, E. P., & Wilson, P. (2014). Best Practices for Scientific Computing. *PLoS Biology*, 12(1), Artículo e1001745. <https://doi.org/10.1371/journal.pbio.1001745>
- Wilson, G., Bryan, J., Cranston, K., Kitzes, J., Nederbragt, L., & Teal, T. K. (2017). Good enough practices in scientific computing. *PLOS Computational Biology*, 13(6), Artículo e1005510. <https://doi.org/10.1371/journal.pcbi.1005510>
- Wu, F., Zhao, S., Yu, B., Chen, Y.-M., Wang, W., Song, Z.-G., Hu, Y., Tao, Z.-W., Tian, J.-H., Pei, Y.-Y., Yuan, M.-L., Zhang, Y.-L., Dai, F.-H., Liu, Y., Wang, Q.-M., Zheng, J.-J., Xu, L., Holmes, E. C., & Zhang, Y.-Z. (2020). A new coronavirus associated with human respiratory disease in China. *Nature*, 579(7798), 265-269. <https://doi.org/10.1038/s41586-020-2008-3>
- Ziemann, M., Eren, Y., & El-Osta, A. (2016). Gene name errors are widespread in the scientific literature. *Genome Biology*, 17(1), Artículo 177. <https://doi.org/10.1186/s13059-016-1044-7>